READ ONCE, ANSWER MANY: PARALLEL PROMPTING FOR SHARED-CONTEXT LLM INFERENCE

Zekun Zhao [1] **Jeffrey Flanigan** [1]

ABSTRACT

LLM workloads sometimes ask many independent questions about the same long document, as in document QA, data labeling and LLM-based evaluation. Prefix caching removes the redundant prefill shared by these queries, but each decode step still reads the document's key–value (KV) cache once per question, so decoding is bound by memory bandwidth and scales with the number of questions. We present *Parallel Prompting*, which decodes P questions as P independent streams of a single request all at once, fixing this issue. In our method, every step reads the shared document once for all P streams, and a stream-id attention mask, evaluated inside the attention kernel, keeps each answer conditioned only on the document and its own question. We implement Parallel Prompting in SGLang by adding about 80 lines to three existing files and a small engine package; the mask needs no dense tensor, and every decode step has a fixed shape that is captured as a CUDA graph. With CodeLlama-7B, 8k–16k-token documents and 64–256 questions per document, Parallel Prompting is **3.7× faster** than stock SGLang with its FlashInfer backend on an A100 and **5.9–6.0× faster** on an RTX A6000, on two SGLang releases, including the most recent release. Timed on identical inputs, it is faster than stock SGLang in all 144 runs (2 GPUs × 2 releases × 6 workloads × 6 input sets). Code: `https://github.com/jlab-nlp/parallel-prompting`. Project page: `https://jlab-nlp.github.io/parallel-prompting/`.

1 INTRODUCTION

Large language models (LLMs) power chat assistants, coding agents, analysis engines and domain-specific agents, and the cost of inference has become a major bottleneck for deploying them (Kwon et al., 2023). Many of these requests share a large part of their input, a *shared prefix*: long system prompts (Zhu et al., 2024), safety policies (Inan et al., 2023), retrieved documents (Lewis et al., 2020), few-shot exemplars or domain knowledge (Wu et al., 2023). In document question answering, large-scale labeling and LLM-based evaluation, tens to hundreds of questions are asked about the same long document (Pang et al., 2022; Wang et al., 2021; Gilardi et al., 2023; Zheng et al., 2023; Liu et al., 2023).

Existing serving engines already exploit this structure. PagedAttention (Kwon et al., 2023) and SGLang's RadixAttention (Zheng et al., 2024) store the document's KV cache once and share it across requests, so the document is prefilled only once. Decoding, however, is still the bottleneck: each question is its own request, and each decode step of each request reads the *whole* document's KV cache from GPU memory again. With Q questions about a document of L tokens, every decode step therefore moves $Q \cdot L$ tokens' worth of KV, and the step is limited by memory bandwidth rather than compute. An alternative, *batch prompting* (Cheng et al., 2023; Lin et al., 2024), puts all questions into one prompt and decodes all answers in one sequence. It reads the document once, but the answers attend to each other, which causes cross-question interference and can degrade quality.

We propose **Parallel Prompting**, which keeps the efficiency of batch prompting and the independence of per-question requests. It places P questions in one request and decodes them as P *streams*: each step appends one token per stream, and a stream-id mask lets a token attend only to the document and to earlier tokens of its own stream. Each stream's positions start right after the document, as if its question were alone. The document's KV is then read roughly once per request per step instead of once per question, which turns P matrix–vector attention products into one matrix–matrix product.

We build Parallel Prompting into SGLang (Zheng et al., 2024). The stream-id mask is evaluated inside SGLang's existing Triton (Tillet et al., 2019) extend-attention kernel from two fixed-size index tensors, so no dense mask is built and every decode step is captured as a CUDA graph. The change adds about 80 lines to three SGLang files plus a small engine package, and we ported it to two releases. Against stock SGLang with prefix caching and CUDA graphs, Parallel Prompting is 3.7× faster than the FlashInfer (Ye et al., 2025) backend on an A100 and 5.9–6.0× faster on an RTX

[1]University of California, Santa Cruz. Preprint.

A6000, and 5.1–7.5× faster than the Triton backend (Figure 3). These figures are for CodeLlama-7B, which uses multi-head attention; on grouped-query-attention models the gain is 1.3–1.9× (subsection 5.6).

Our contributions are:

- We propose *Parallel Prompting*, which decodes many questions about one document as independent streams of a single request (section 2 and section 3).
- We implement it in a production engine, SGLang, with an in-kernel stream-id mask that stays compatible with CUDA graphs, and port it across two releases (section 4).
- We evaluate it with paired, seeded measurements on two GPUs, two SGLang versions and six models. On the main grid it is faster in all 144 runs, each timed on the same inputs as stock SGLang. We also show how the gain depends on document length and question count, how it shrinks with long answers, and how to choose P(section 5).

2 Background and Motivation

Attention in prefill and decode. Given queries $\mathbf{Q} \in \mathbb{R}^{N_q \times d}$, keys $\mathbf{K} \in \mathbb{R}^{N_{kv} \times d}$ and values $\mathbf{V} \in \mathbb{R}^{N_{kv} \times d}$, attention computes

$$\mathbf{O} = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}} + \mathbf{M}\right)\mathbf{V}, \tag{1}$$

with a causal mask $\mathbf{M}$. During prefill, the whole prompt is processed in one forward pass ($N_q = N_{kv} \gg 1$), and attention is a compute-friendly matrix–matrix product. During decode, each step processes one new token per sequence against a KV cache of all earlier tokens ($N_q = 1$, $N_{kv} \gg 1$). Attention becomes a matrix–vector product whose cost is dominated by reading $\mathbf{K}$ and $\mathbf{V}$ from memory. Batching many sequences helps the MLP layers, which share weights, but not attention, because each sequence has its own KV cache.

Shared prefixes under prefix caching. Consider a document D of L tokens and questions $q_1, \ldots, q_Q$, where each answer should be $a_i = \pi_{\mathrm{LLM}}(D, q_i)$. A serving engine with prefix caching prefills D once and points every request's KV table at the same cached pages. This saves compute and memory, but each decode step of each request still attends over all L document tokens. A step over Q requests therefore reads the document's KV Q times. For long documents this term dominates the memory traffic of a decode step, and it grows linearly in both L and Q. Shared-prefix attention kernels (Juravsky et al., 2024; Zhu et al., 2024; Ye et al., 2024; Athiwaratkun et al., 2024; Ye et al., 2025; Pan et al., 2025) attack this term with a dedicated kernel that computes the prefix part of attention for many requests at once and merges it with each request's suffix attention.

Batch prompting. Batch prompting (Cheng et al., 2023; Lin et al., 2024) concatenates all questions after D and lets the model write all answers in one sequence. It reads D once per step, but the hidden state for answer i contains every earlier question and answer. Answers are no longer independent, and quality can degrade; our prototype shows this on SQuAD and DROP (Table 9 in Appendix D). Figure 1 contrasts batch prompting with batched requests and with Parallel Prompting.

3 Parallel Prompting

Parallel Prompting generates all answers in parallel while keeping each answer conditioned only on (D, q_i). A request holds the document followed by P questions, and is decoded as P streams:

$$\texttt{[D] [q}_1\texttt{]} \ldots \texttt{[q}_P\texttt{][a}_1^1 \ldots \texttt{a}_P^1\texttt{][a}_1^2 \ldots \texttt{a}_P^2\texttt{]} \ldots$$

Here a_s^t is the t-th token of stream s.

Stream ids and the mask. Every KV slot carries a stream id sid: -1 for document tokens and s for tokens of question s and its answer. A token of stream s may attend to a key k only if

$$\mathrm{causal}(q, k) \;\wedge\; \big(\mathrm{sid}_k = -1 \;\vee\; \mathrm{sid}_k = \mathrm{sid}_q\big). \tag{2}$$

Every stream therefore sees exactly the tokens it would see if its question were decoded alone.

Positions. Each stream's positions start at L, right after the document, so rotary embeddings match those of a single-question request.

Prefill. If D is already cached, only the P questions are prefilled, as one packed sequence under Equation 2, in the manner of prepacking (Zhao et al., 2025). Otherwise D is prefilled once and shared by every request that uses it.

Parallel decode. Each decode step appends one token to every stream: P query tokens per request, all attending to the same document KV. The document's keys and values are loaded once per query tile, instead of once per question. The P matrix–vector products of standard decoding thus become one matrix–matrix product with P rows. The output logits are sampled independently, and finished streams are masked out until the request ends.

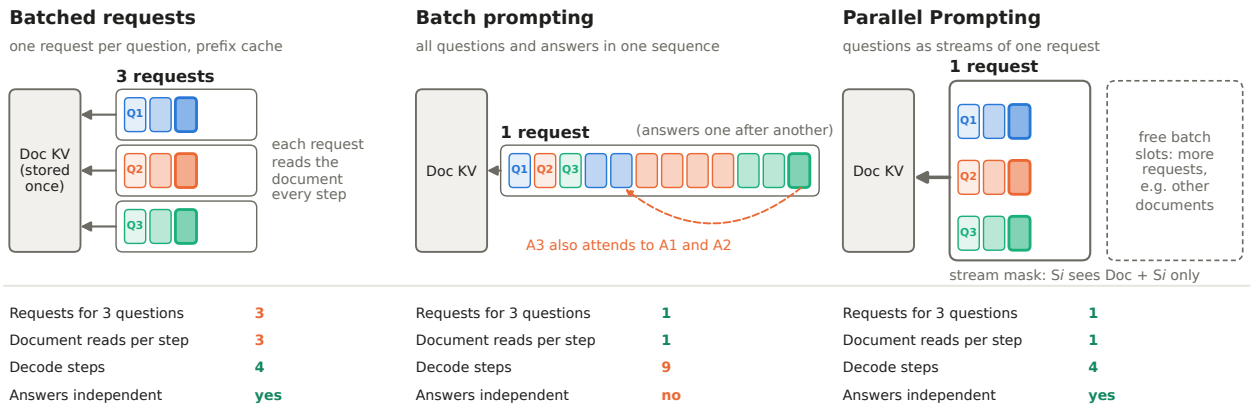


Figure 1: Three ways to answer several questions about one document, shown at one decode step (dark: the token generated in this step). A serving engine such as SGLang or vLLM runs a *batch* of *requests*, and every decode step adds one token to each request. **Batched requests** with prefix caching store the document's KV once but use one request per question, so every step reads the document once per question. **Batch prompting** (Cheng et al., 2023; Lin et al., 2024) puts all questions and answers in one sequence: it reads the document once, but the answers are generated one after another, and later answers attend to earlier ones. **Parallel Prompting** decodes the questions as *streams* of one request: every step adds one token per stream and reads the document once for all of them, and a stream mask (Figure 2) keeps the answers independent. The batch still has room for other requests, for example for other documents (Figure 6).

Batching and choosing P. Questions about one document are split into $\lceil Q/P \rceil$ requests, and requests over different documents are batched together as usual. P trades attention efficiency against the extra keys each stream must skip: the KV row of a request also holds the other streams' tokens, which the kernel scans and masks out. Our analysis in Appendix C suggests that throughput is maximized at an interior point. Writing B for the batch size, the number of requests per step, throughput is

$$\mathrm{Thr}(P, B) = \frac{P\,B}{T_{\mathrm{attn}}(P) + T_{\mathrm{mlp}}(PB)}, \qquad (3)$$

where T_{attn} falls with P until the extra masked keys dominate, and T_{mlp} grows with the number of tokens, but sublinearly until the GPU is saturated. In particular, the largest P does not maximize Equation 3. In subsection 5.4, we show that the optimum is broad and similar across GPUs and engine versions. Algorithm 1 summarizes the generation loop.

4 Implementation in SGLang

We implemented Parallel Prompting in SGLang v0.5.22 and v0.4.2.post3. Table 1 lists our changes. Every edit is inactive unless a Parallel Prompting batch turns it on, so stock SGLang takes its original code path.

In-kernel mask. SGLang's Triton *extend* kernel attends a batch of new tokens to cached tokens (stage 1) and to themselves (stage 2). We add two integer tensors: q_sid, the stream id of each new token, and slot_sid, the stream id of each KV slot. In both stages, the kernel loads the two tensors and combines the condition of Equation 2 with its causal mask (Figure 2). A dense tokens $\times$ KV mask would grow with the KV row and change size every step. The two index tensors have a fixed size per step, so a step can be captured as a CUDA graph. For decode steps, which have only P new tokens per request, we use a smaller query tile.

Table 1: Modifications based on SGLang v0.5.22. The three edited files gain 79 lines.

File	Change	Lines
`extend_attention.py`	stream-id mask in both kernel stages; small tile for decode steps	+50
`triton_backend.py`	pass `q_sid`, `slot_sid` to the kernel	+11
`logits_processor.py`	one logit row per stream (`logit_index`)	+18
`parallel_prompt/`	new: layout, reference mask, engine	new

Engine. Parallel Prompting runs as an engine on SGLang's `ModelRunner`. It prefills each document once, then gives each request a row in the request-to-token table that begins with the document's cached slots, so the document is shared without copying. Questions are split into requests of P streams and run in batches sized to the free

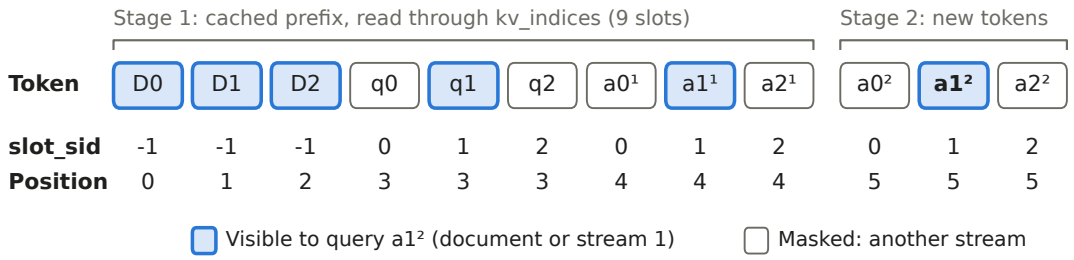


Figure 2: The in-kernel mask in one decode step: a request with a 3-token document, $P = 3$ streams and 1-token questions, at decode step 2 (a_i^k is stream i's k-th answer token). Each KV slot carries its stream id $\mathrm{slot_sid}$ (-1 for the document), and the streams share positions. The new token a_1^2 of stream 1 attends only to the document and to stream 1's slots. Stage 1 of the kernel reads the cached slots through `kv_indices`; stage 2 reads the step's new tokens.

KV pool. Every step, prefill or decode, is an ordinary EXTEND batch. One CUDA graph is captured for each pair (B, P), where B is the number of requests in a batch and P the number of streams per request. B is rounded to a power of two, so batches of similar size reuse one graph. The logits processor returns the P last-token rows of each request, one per stream. In terms of Algorithm 1, PREPAREMASK becomes the two index tensors, and each FORWARD call becomes one EXTEND step over all requests of a batch.

Continuous batching. By default, the engine runs each batch to completion, so a batch waits for its slowest stream. In the continuous-batching variant, the engine checks for early finished requests and never lets a request pass its answer budget. A request whose streams have all stopped leaves the batch, its slots return to the KV pool, and waiting requests are prefilled into the free places. Each request keeps its own step counter, so requests in one batch can be at different answer positions. The batch is padded to a power of two with a dummy request, so the CUDA graph for each (B, P) is reused.

Correctness checks. A CPU test compares the layout and positions with a reference implementation of Equation 2. A kernel test compares the Triton output with PyTorch scaled-dot-product attention under the dense reference mask. On three layouts (documents of 37, 300 and 1,000 tokens with 1 to 64 streams), covering prefill and four decode steps, the largest absolute error in bf16 is 0.009, below the test's tolerance of 0.03. An end-to-end check compares Parallel Prompting's answer quality with stock SGLang's (Appendix B).

5 EVALUATION

We compare Parallel Prompting with stock SGLang and answer four questions:

1. How much faster is Parallel Prompting than a tuned production engine (subsection 5.2)?
2. Which workloads benefit, and which do not (subsection 5.3)?
3. How should the number of streams per request P be chosen (subsection 5.4)?
4. How fast and how accurate is it on real long-document benchmarks, with prefix caching and continuous batching (subsection 5.5)?

We then repeat the main experiments on other models (subsection 5.6) and summarize the results of our HuggingFace prototype (subsection 5.7).

5.1 Setup

Systems. We compare three configurations of the same SGLang build on the same GPU:

- **Stock FlashInfer:** stock SGLang with prefix caching, the FlashInfer attention backend (Ye et al., 2025) and CUDA graphs. It is SGLang's default backend and the faster stock configuration in most of our settings (subsection 5.2).
- **Stock Triton:** the same engine with the Triton backend. It shares the attention backend with Parallel Prompting, but stock decoding uses the backend's decode kernel, while Parallel Prompting runs every step through the extend kernel.
- **Parallel Prompting (PP):** our engine on the Triton backend with CUDA graphs.

For the stock configurations, the document is cached before timing, and CUDA graphs are captured for the full decode batch. We raise SGLang's CUDA-graph batch-size limit

Algorithm 1 Parallel Prompting generation loop (prototype formulation with explicit masks; the SGLang engine evaluates the mask inside the kernel, Section 4).

Require: Shared prefix `Doc`, unique suffix set `Q_all`, questions per batch `N`, streams per request `P`, language model π_{LLM}
Ensure: List of generated answers

Optional: `cache` ← PRECOMPUTE(π_{LLM}, `Doc`) ▷ Prefill KV-cache for shared prefix
`i` ← 0
`N_p` ← `N` / `P` ▷ Requests per batch (B)
while `i` < |`Q_all`| **do**
  `Q_n` ← `Q_all[i : i + N]`
  `Q_np` ← PARALLELIZEINTERLEAVE(`Q_n`, `P`)
  `prompts` ← PREPAREINPUT(`Doc`, `Q_np`, `N_p`)
  `masks` ← PREPAREMASK(`prompts`)
  `answers`, `output_mask` ← PARALLELGENERATE(π_{LLM}, `prompts`, `masks`, `P`, `cache`)
  for `n` = 1 **to** `N_p` **do**
    for `p` = 1 **to** `P` **do**
      `final_answer.append(DECODE(answers[n, p], output_mask[n, p]))`
    end for
  end for
  `i` ← `i + N`
end while
return `final_answer`

function PARALLELGENERATE(π_{LLM}, `prompts`, `masks`, `P`, `cache`)
  `finished` ← `False`
  `input_ids` ← TOKENIZE(`prompts`)
  while not `finished` **do**
    `outputs` ← π_{LLM}.FORWARD(`input_ids`, `masks`, `cache`)
    `logits` ← `outputs[:, -P:]` ▷ Outputs `P` logits on sequence dimension
    `next_tokens` ← SAMPLE(`logits`)
    `input_ids` ← CONCAT(`input_ids`, `next_tokens`)
    if STOPPINGCRITERIA(`input_ids`) **then**
      `finished` ← `True`
    else
      `masks` ← UPDATEPARALLELMASK(`input_ids`, `P`)
    end if
  end while
  return `input_ids`, `masks`
end function

function PRECOMPUTE(π_{LLM}, `Doc`)
  `kv_cache` ← π_{LLM}.FORWARD(`Doc`)
  return `kv_cache`
end function

to the full decode batch, because the default limit can be smaller, which would understate stock performance. On the real-data benchmarks (subsection 5.5), every configuration uses SGLang's radix prefix cache. **PP** also inserts each document into the radix cache, so a prefix shared by several documents, such as the instruction, is prefilled once. A separate study measures **PP + continuous batching (PP+CB)**, which adds the scheduling described in section 4: requests leave the batch as they finish, and waiting requests take their places.

Hardware and software. We use an NVIDIA A100-SXM4-80GB (2.0 TB/s) and an RTX A6000 (48 GB, 0.77 TB/s), each running SGLang v0.4.2.post3 and v0.5.22. The main synthetic grid uses CodeLlama-7B-Instruct (Rozière et al., 2024), and subsection 5.6 repeats it on five more models; all runs use bf16 and greedy decoding. The real-data benchmarks use Qwen2.5-7B-Instruct (Qwen Team, 2024) with SGLang v0.5.22, on the A100 except for the continuous-batching study (Table 4), which runs on the A6000. The default number of generated tokens is 128 per question, unless stated otherwise.

Data. Following Juravsky et al. (2024), documents are drawn from *War and Peace* (Tolstoy, 1869) and questions are generated per document. Each document is a different slice of the book, chosen by a random seed, and the i-th question comes from a fixed template, "Summarize the main point of paragraph i.", so all questions of a document have nearly the same length. The main grid uses 8k- and 16k-token documents with 64, 128 or 256 questions per document.

Real long-document benchmarks. We also use two public benchmarks in which many questions share a long document. **QuALITY** (Pang et al., 2022) (v1.0.1, dev split) has 2,086 four-way multiple-choice questions about 115 articles, 18 questions per article on average, with articles of 5.7k tokens on average. Our prompt asks for the letter of the answer, with at most 16 answer tokens, and we report accuracy. **LooGLE** (Li et al., 2024) (short-dependency QA, test split) has 1,761 free-form questions about 96 of its 105 Wikipedia documents that fit a 32k-token context, 18 questions per document on average, with documents of 19.9k tokens on average. We use its official prompt and answer budget of 300 tokens, and report token recall, the fraction of the reference answer's words that appear in the model's answer, which tracks human judgments of instruction-following models better than F1 (Adlakha et al., 2024). In both, the shared document is the instruction plus the document, each question is one stream, and decoding is greedy until the end-of-sequence token or the answer budget. Calibration uses questions outside the scored set: QuALITY's training split and LooGLE's long-dependency questions.

Methodology. Every comparison is *paired*. A *seed* is the random seed that draws one instance of a workload, that is, its documents and questions; the same seed is timed under every configuration on the same GPU. We measure *wall time*: the time to answer all questions of one seed, from submitting them until the last answer is returned, with the

document already cached. Each workload uses six evaluation seeds, run in two passes with the configuration order reversed, plus a warm-up seed and a calibration seed. PP's P is chosen on the calibration seed. We report the geometric mean of per-seed wall-time ratios with a 95% bootstrap confidence interval. Stock SGLang is driven through its offline `Engine` API in the same process. Before timing, a one-token request per document fills the radix cache; the Q requests (document plus question, already tokenized) are then submitted in a single `generate` call, with SGLang's default limit on running requests. PP runs on the same `ModelRunner` through our engine, with the documents prefilled before timing. Each workload uses the same static memory fraction for every configuration (0.8 by default; 0.88 in the model sweep and some A6000 runs). For every configuration, the timed wall time includes question prefill and all decode steps, and excludes tokenization of the inputs. The real-data benchmarks are timed differently, because their documents are part of the work: each pass starts with an empty prefix cache, and the wall time covers document and question prefill and all decode steps for the whole benchmark. Stock SGLang receives one request per question, all in one `generate` call, so its radix cache can share a document's prefill among that document's questions. Every configuration first runs a warm-up on the calibration questions, then runs one untimed pass to capture its CUDA graphs. We report the mean of two passes, and all configurations of a table run back to back on one GPU.

Memory. We do not compare whole-GPU memory between PP and stock SGLang, because it does not reflect the workload. SGLang reserves its KV-cache pool when it starts, as a fixed fraction of GPU memory (`mem_fraction_static`). PP and stock run with the same value in every comparison, so `nvidia-smi` reports nearly the same memory for both. Our HuggingFace prototype allocates memory on demand, so for it we report peak GPU memory: PP uses less memory than standard prompting on SQuAD (Rajpurkar et al., 2016), QuAC (Choi et al., 2018), and DROP (Dua et al., 2019) (Table 10 in Appendix E), and fewer questions per batch lower it further (Table 6 in subsection 5.7).

5.2 Speedup over a production engine

Figure 3 summarizes the main grid. Against stock FlashInfer, PP is 3.7× faster on the A100 (v0.4.2 and v0.5.22) and 5.9–6.0× faster on the A6000. Against stock Triton, which shares PP's attention backend, it is 5.1–7.5× faster. PP wins all 144 paired runs. The ranges do not overlap: on the v0.4.2 A100 setting, the slowest PP run beats the fastest stock run of the same workload by 2.46–4.76× (one-sided sign test against the best competitor, $p = 1.5 \times 10^{-11}$). Table 2 gives the absolute times. For example, PP answers 256 questions about a 16k-token document in 12.3 s, where stock SGLang needs 60.0 s.

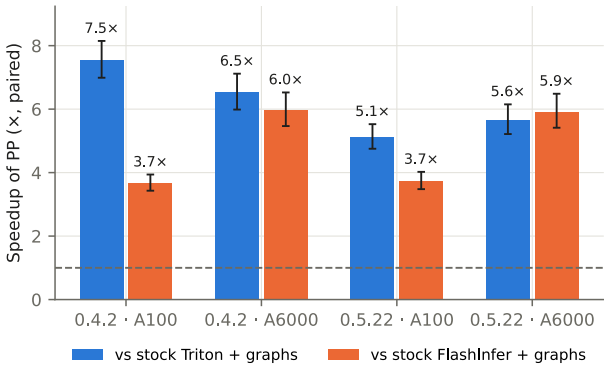


Figure 3: Speedup of Parallel Prompting over stock SGLang on 8k/16k-token documents with 64–256 questions. Error bars are 95% bootstrap CIs over 36 paired runs per setting. PP wins all 144 runs.

Table 2: Wall time (s, mean of 6 seeds) on SGLang v0.4.2 with an A100-80GB, and PP's speedup over the faster stock configuration. Wall time is the time to answer all Q questions about one document, which is already cached (subsection 5.1). FI = FlashInfer; all configurations use CUDA graphs.

Doc	Q	FI	Triton	PP (P)	Speedup
8k	64	9.76	18.19	3.54 (16)	2.72×
8k	128	16.19	34.44	4.66 (32)	3.47×
8k	256	32.26	68.07	8.30 (32)	3.89×
16k	64	16.17	32.61	5.29 (16)	3.06×
16k	128	30.61	63.55	6.80 (32)	4.50×
16k	256	59.99	126.15	12.31 (32)	4.87×

5.3 Which workloads benefit

Document length and question count. The gain grows with the amount of document KV that stock decoding rereads (Figure 4, left). With 16 questions about a 512-token document, PP and stock SGLang tie (0.99–1.04×). With 256 questions about a 16k-token document, PP is 5.0× faster on the A100 (v0.5.22) and 7.4–8.5× faster on the A6000.

Output length. Long answers shrink the gain (Figure 4, right). From 128 to 1024 output tokens, the speedup falls from 3.6× to 1.9× on the A100 (v0.5.22) and from 5.2–5.4× to 1.9–2.0× on the A6000, though PP stays faster. We attribute this to our interleaved layout. A request's KV row also holds its sibling streams' generated tokens, so at answer length t each stream scans about $P \cdot t$ extra keys that the mask then discards.

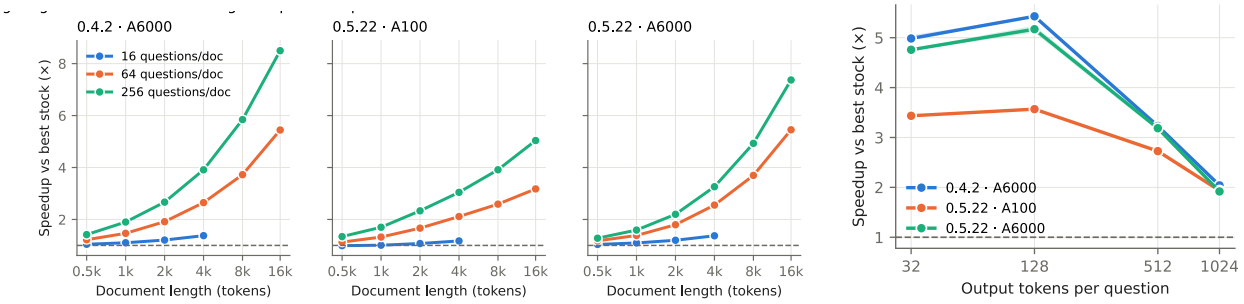


Figure 4: **Left:** speedup over the faster stock configuration as document length and questions per document grow (128 output tokens). **Right:** speedup as answers get longer (8k-token document, 128 questions).

Table 3: QuALITY and LooGLE on an A100-SXM4-80GB with Qwen2.5-7B-Instruct and SGLang v0.5.22, every configuration with the radix prefix cache. Time (s) is the wall time for the whole benchmark from an empty cache, including document prefill (mean of two passes, all rows on one GPU). LooGLE is scored by token recall; token F1 is 17.29, 17.41 and 17.36.

	QuALITY		LooGLE	
Config	Time	Acc	Time	Recall
Stock FlashInfer	133.2	76.13	403.6	83.41
Stock Triton	155.9	76.22	568.0	83.77
PP	73.6	75.89	508.3	83.61

Many documents per batch. Batching requests from several documents increases the gain. On v0.5.22 with an A100, 4k-token documents and 64 questions each, PP is 2.1× faster with one document per batch, 2.9× with four and 3.3× with sixteen. On the A6000, the speedup is 2.6–2.7× with one document and 3.4–3.7× with four; sixteen documents do not fit in its KV pool (Figure 6 in Appendix A).

5.4 Choosing P

Wall time is U-shaped in P, as Equation 3 predicts (Figure 5). Small P leaves the document re-read many times. Large P makes each stream scan many masked sibling tokens and lengthens each step. The best P is 16 for 64 questions and 32 for 128 or 256 questions, on both GPUs and both versions; the one exception is $P = 64$ for 256 questions on v0.5.22 with the A100. Near the optimum the curve is flat: with 256 questions about a 16k-token document on the A100, $P = 32$ and $P = 64$ differ by less than 1%. A fixed default of $P = 32$ stays within about 12% of the best P in every setting.

Table 4: Continuous batching on QuALITY in the chat template (Qwen2.5-7B-Instruct, RTX A6000, $P = 4$, 512-token budget, mean of two passes). *Uniform*: every answer is the letter (2 tokens). *Mixed*: every other article asks for an explanation first (87 tokens on average, at most 169).

	Uniform		Mixed	
Config	Time	Steps	Time	Steps
PP	161.0	752	356.5	6,160
PP+CB	161.0	752	292.1	4,176
Speedup	1.00×		1.22×	

5.5 Real long-document benchmarks

Table 3 runs the configurations of subsection 5.1 on two real benchmarks, timing the whole benchmark from an empty cache. On QuALITY, whose answers are short, PP answers all 2,086 questions in 73.6 s, 1.81× faster than stock FlashInfer and 2.12× faster than stock Triton. On LooGLE, PP is 1.12× faster than stock Triton, which uses the same attention kernels, but 1.26× slower than stock FlashInfer. LooGLE's answers are long, 234 tokens on average, which shrinks PP's gain because of the interleaved layout (subsection 5.3). And on these 20k-token documents, FlashInfer makes stock SGLang 1.41× faster than Triton, a kernel advantage that PP, which runs only on Triton, does not get.

Continuous batching. Continuous batching can only help when some requests finish earlier than others while more requests wait. That does not happen in either benchmark above: every QuALITY answer runs to its 16-token budget, and each LooGLE batch already holds all of its requests, so none are waiting and PP+CB matches PP there.

To isolate the effect, we rerun QuALITY in the model's chat template, so that answers end with the end-of-sequence token, in two versions (Table 4). When every answer is the letter alone, all requests finish on the same step, and PP+CB takes exactly as long as PP. When every other article asks

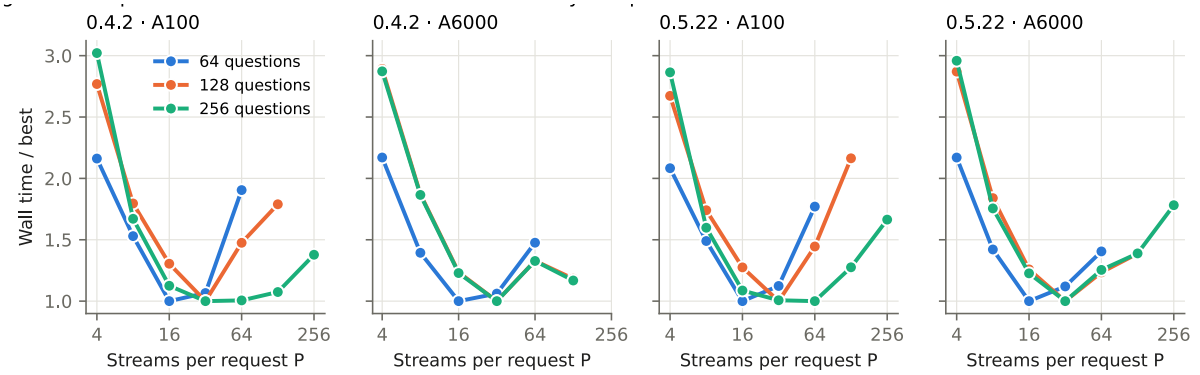


Figure 5: Wall time versus streams per request P on the calibration seed, normalized to the best P (16k-token documents). The curve is U-shaped, with its minimum at P = 16–64 on every setting.

Table 5: PP speedup across models on SGLang v0.5.22 (paired geometric mean over 6 seeds per workload, 128 output tokens). KV/token is the KV-cache size per token in bf16. Workloads are 8k and 16k-token documents with 64 and 256 questions, except where noted.

Model	Attn.	KV/tok	GPU	vs FI	vs Triton	Wins
Qwen2.5-1.5B	GQA	28 KB	A6000	1.28×	1.42×	12/24
Qwen2.5-7B	GQA	56 KB	A6000	1.32×	1.51×	24/24
Qwen2.5-14B	GQA	192 KB	A6000	1.87×	2.08×	24/24
Qwen2.5-32B†	GQA	256 KB	A100	1.26×	1.49×	12/12
CodeLlama-7B‡	MHA	512 KB	A6000	5.91×	5.65×	36/36
CodeLlama-13B†	MHA	800 KB	A6000	3.54×	3.42×	12/12

†8k documents only (KV pool). ‡Main grid of subsection 5.2, including 128 questions.

for an explanation before the letter, PP waits in each batch for its longest answer, while PP+CB refills the freed places with waiting requests: it needs 32% fewer decode steps and is 1.22× faster, with accuracy within 0.2 points (75.55 vs. 75.36). On the mixed workload, this cuts decode steps from 6,160 to 4,176.

Answers. Accuracy and recall agree within 0.33 points on QuALITY and 0.36 on LooGLE across all configurations. On SQuAD v1.1, PP's EM and F1 also match both stock backends, and every paired 95% confidence interval on the F1 difference contains zero (Appendix B).

5.6 Other model sizes

Table 5 repeats the main grid on five more models, including Qwen2.5 (Qwen Team, 2024), which uses grouped-query attention (GQA). The gain tracks how much KV each stock decode step re-reads. With full multi-head attention, CodeLlama stores 512–800 KB of KV per token, and PP is 3.4–5.9× faster. GQA shrinks the KV per token, so stock decoding re-reads far less; with 28–56 KB per token, Qwen2.5-1.5B and 7B gain 1.3–1.5×. On Qwen2.5-1.5B, the smallest KV in our study, PP is 1.6–1.9× faster with 256 questions per document but 5–7% slower than stock FlashInfer with 64; there, re-reading a 28 KB-per-token document is cheap, and PP's extra masked keys cost more than they save. CodeLlama-13B on a 48 GB GPU leaves a KV pool of only about 20k tokens for stock SGLang, which must then run the batch in small pieces. Its 3.5× therefore reflects PP's smaller KV footprint as well as its kernel savings.

The larger Qwen models follow the same trend: Qwen2.5-14B wins all 24 paired runs (1.87× over FlashInfer, from 1.45× at 8k/64 to 2.48× at 16k/256), helped by a tight 76k-token KV pool on 48 GB, and Qwen2.5-32B on an A100 wins all 12 at 1.26×. For every model, the speedup grows with the number of questions, and for the four models run at both document lengths it also grows with document length.

5.7 HuggingFace Implementation

Along with the SGLang implementation, we built Parallel Prompting on HuggingFace Transformers (Wolf et al., 2020), with a dense stream mask passed to the model. The prototype allowed comparisons with methods that have no SGLang implementation, including Hydragen (Juravsky et al., 2024) and batch prompting, and an evaluation of answer quality on reading-comprehension tasks. With 512-token documents the prototype matched Hydragen's throughput; at 1024 and 2048 tokens, Hydragen's dedicated prefix kernel was faster (Table 8 in Appendix D), which motivated moving Parallel Prompting into an engine with fused kernels. Running on A100-80GB with Llama 3–8B, the prototype is 3.5× faster than standard prompting on SQuAD and 5.5× on DROP at equal F1.

Memory. The HuggingFace prototype allocates memory on demand, so its peak memory shows how PP's footprint

Table 6: Peak GPU memory of the HuggingFace prototype on QuAC (Llama 3–8B, A100-80GB) as the maximum number of questions per batch, across documents, varies. Standard prompting asks one question per prompt, with batches as large as memory allows; its row comes from a separate run. Speedup is over standard prompting.

Method	Q/batch	Time (s)	Speedup	Mem (GiB)	F1
Standard	32(max)	1799	1.0×	53.6	33.99
PP	8	872	2.1×	16.9	33.96
PP	16	677	2.7×	19.8	33.98
PP	32	420	4.3×	24.7	34.05
PP	64	352	5.1×	33.1	33.87
PP	128	342	5.3×	54.1	33.92

scales with the batch. Table 6 varies the maximum number of questions per batch on QuAC: memory grows with it, while F1 stays within 0.2 points. With at most 8 questions per batch, PP uses 16.9 GiB, less than a third of standard prompting's 53.6 GiB, and is still 2.1× faster; with 32, it is 4.3× faster with 46% of the memory. On SQuAD and DROP, at the largest batch that fits, PP's peak memory is 13% and 34% lower than standard prompting's (Table 10 in Appendix E).

6 Related Work

Prefix KV reuse. PagedAttention (Kwon et al., 2023) in vLLM and RadixAttention (Zheng et al., 2024) in SGLang share cached prefix KV across requests, and prompt caching (Gim et al., 2024) reuses precomputed KV for prompt modules. These methods remove redundant prefill and memory, but each request still reads the shared KV on every decode step. Parallel Prompting builds on prefix caching and targets that remaining read.

Shared-prefix attention. Hydragen (Juravsky et al., 2024), RelayAttention (Zhu et al., 2024), ChunkAttention (Ye et al., 2024) and FlashInfer's cascade attention (Ye et al., 2025) split attention into a shared-prefix part, computed for many requests as one matrix–matrix product, and a per-request suffix part, merged afterwards. Bifurcated attention (Athiwaratkun et al., 2024) applies the same split to many samples from one context. DeFT (Yao et al., 2025) extends this to tree-structured sharing, and FastTree (Pan et al., 2025) groups queries that share a radix-tree prefix inside SGLang and runs them through dedicated Triton kernels. These approaches need a dedicated kernel and a split or tree-structured KV layout. Parallel Prompting instead changes how requests are formed: the shared-prefix reuse comes from packing streams into one request, so it reuses the engine's existing extend kernel, KV format and CUDA graphs, with a mask added. We expect the two ideas to be complementary, since a split-prefix kernel could serve each packed request, but we have not measured the combination.

Packing and batch prompting. Prepacking (Zhao et al., 2025) packs prompts of different lengths into one sequence with a block-diagonal mask, for prefill only. SGLang's multi-item scoring packs several candidates after a shared query, with a mask that keeps the items apart, also for prefill only.[1] PDOS (Yu et al., 2025) decodes parallel reasoning branches inside one sequence with a tree mask and shared position ids, and concurrent work, IPPD (Glavas et al., 2026), answers several questions over a shared context in one prompt with virtual position ids and an attention mask, implemented in HuggingFace Transformers. Parallel Prompting applies the same principle to independent questions over a long shared document inside a production engine, with an in-kernel mask that keeps CUDA graphs, and measures the decode-bandwidth savings against the engine's tuned stock configurations. Batch prompting (Cheng et al., 2023; Lin et al., 2024) shares the prompt but lets answers interact, which can harm quality.

Faster decoding. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023; Miao et al., 2024) and multi-token prediction (Cai et al., 2024; Gloeckle et al., 2024) speed up a single sequence. They are orthogonal to Parallel Prompting and could run within each stream. Other work reduces KV traffic through sparsity (Sun et al., 2024) or offloading (Sheng et al., 2023).

Prefix-aware scheduling. BatchLLM (Zheng et al., 2026) identifies shared prefixes globally in large offline batches and schedules requests that share them together. Such schedulers increase prefix sharing; Parallel Prompting reduces the cost of each shared decode step, so the two are complementary.

7 Limitations and Conclusion

Limitations. Our engine targets batch and offline workloads, in which the requests that share a document are known in advance; serving requests as they arrive requires integration with an online scheduler, which we leave to future work. The implementation also supports only a subset of the serving engine's features, such as one attention backend, a single GPU and greedy decoding. The benefit depends on the workload: it is largest when many questions share a long document and answers are short, and it shrinks when answers are long or vary widely in length, when the model stores little KV cache per token, or when the baseline uses faster attention kernels. For such workloads, an engine can fall back to standard batching. Finally, Parallel Prompting changes the order of floating-point operations, so its out-

[1] https://github.com/sgl-project/sglang/pull/10979

puts are not bitwise identical to standard decoding, although task accuracy matches it within a fraction of a point in our experiments.

Conclusion. Prefix caching stops recomputing a shared document but still re-reads it once per question at every decode step. Parallel Prompting decodes many questions as independent streams of one request, so each step reads the document once per request instead of once per question. Parallel prompting is 3.7–6.0× faster than stock SGLang with FlashInfer on long-document, many-question workloads with CodeLlama-7B, and 1.3–1.9× faster on grouped-query-attention Qwen2.5 models from 7B to 32B.

ACKNOWLEDGEMENTS

This work used resources available through the National Research Platform (NRP) at the University of California, San Diego. NRP has been developed, and is supported in part, by funding from National Science Foundation, from awards 1730158, 1540112, 1541349, 1826967, 2112167, 2100237, and 2120019, as well as additional funding from community partners.

REFERENCES

Adlakha, V., BehnamGhader, P., Lu, X. H., Meade, N., and Reddy, S. Evaluating correctness and faithfulness of instruction-following models for question answering. *Transactions of the Association for Computational Linguistics*, 12:681–699, 2024.

Amdahl, G. M. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.

Athiwaratkun, B., Gonugondla, S. K., Gouda, S. K., Qian, H., Ding, H., Sun, Q., Wang, J., Guo, J., Chen, L., Bhatia, P., Nallapati, R., Sengupta, S., and Xiang, B. Bifurcated attention for single-context large-batch sampling. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 1971–1991, 2024.

Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning*, 2024.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Cheng, Z., Kasai, J., and Yu, T. Batch prompting: Efficient inference with large language model APIs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 792–810, 2023.

Choi, E., He, H., Iyyer, M., Yatskar, M., Yih, W.-t., Choi, Y., Liang, P., and Zettlemoyer, L. QuAC: Question answering in context. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 2174–2184, 2018.

Dua, D., Wang, Y., Dasigi, P., Stanovsky, G., Singh, S., and Gardner, M. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 2368–2378, 2019.

Gilardi, F., Alizadeh, M., and Kubli, M. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30): e2305016120, 2023.

Gim, I., Chen, G., Lee, S.-s., Sarda, N., Khandelwal, A., and Zhong, L. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems*, 6:325–338, 2024.

Glavas, T., Vedula, N., Dhyani, D., Valkanas, A., Zhu, Y., and Malmasi, S. Intra-prompt parallel decoding for common-context question answering. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, pp. 58871–58890, 2026.

Gloeckle, F., Idrissi, B. Y., Rozière, B., Lopez-Paz, D., and Synnaeve, G. Better & faster large language models via multi-token prediction. In *International Conference on Machine Learning*, 2024.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Roziere, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Allonsius, D., Song, D., Pintz, D., Livshits, D., Wyatt, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lakomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Guzmán, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Thattai, G., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I., Misra, I., Evtimov, I., Zhang, J., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J.,

Shah, J., van der Linde, J., Billock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnstun, J., Saxe, J., Jia, J., Alwala, K. V., Prasad, K., Upasani, K., Plawiak, K., Li, K., Heafield, K., Stone, K., El-Arini, K., Iyer, K., Malik, K., Chiu, K., Bhalla, K., Lakhotia, K., Rantala-Yeary, L., van der Maaten, L., Chen, L., Tan, L., Jenkins, L., Martin, L., Madaan, L., Malo, L., Blecher, L., Landzaat, L., de Oliveira, L., Muzzi, M., Pasupuleti, M., Singh, M., Paluri, M., Kardas, M., Tsimpoukelli, M., Oldham, M., Rita, M., Pavlova, M., Kambadur, M., Lewis, M., Si, M., Singh, M. K., Hassan, M., Goyal, N., Torabi, N., Bashlykov, N., Bogoychev, N., Chatterji, N., Zhang, N., Duchenne, O., Çelebi, O., Alrassy, P., Zhang, P., Li, P., Vasic, P., Weng, P., Bhargava, P., Dubal, P., Krishnan, P., Koura, P. S., Xu, P., He, Q., Dong, Q., Srinivasan, R., Ganapathy, R., Calderer, R., Cabral, R. S., Stojnic, R., Raileanu, R., Maheswari, R., Girdhar, R., Patel, R., Sauvestre, R., Polidoro, R., Sumbaly, R., Taylor, R., Silva, R., Hou, R., Wang, R., Hosseini, S., Chennabasappa, S., Singh, S., Bell, S., Kim, S. S., Edunov, S., Nie, S., Narang, S., Raparthy, S., Shen, S., Wan, S., Bhosale, S., Zhang, S., Vandenhende, S., Batra, S., Whitman, S., Sootla, S., Collot, S., Gururangan, S., Borodinsky, S., Herman, T., Fowler, T., Sheasha, T., Georgiou, T., Scialom, T., Speckbacher, T., Mihaylov, T., Xiao, T., Karn, U., Goswami, V., Gupta, V., Ramanathan, V., Kerkez, V., Gonguet, V., Do, V., Vogeti, V., Albiero, V., Petrovic, V., Chu, W., Xiong, W., Fu, W., Meers, W., Martinet, X., Wang, X., Wang, X., Tan, X. E., Xia, X., Xie, X., Jia, X., Wang, X., Goldschlag, Y., Gaur, Y., Babaei, Y., Wen, Y., Song, Y., Zhang, Y., Li, Y., Mao, Y., Coudert, Z. D., Yan, Z., Chen, Z., Papakipos, Z., Singh, A., Srivastava, A., Jain, A., Kelsey, A., Shajnfeld, A., Gangidi, A., Victoria, A., Goldstand, A., Menon, A., Sharma, A., Boesenberg, A., Baevski, A., Feinstein, A., Kallet, A., Sangani, A., Teo, A., Yunus, A., Lupu, A., Alvarado, A., Caples, A., Gu, A., Ho, A., Poulton, A., Ryan, A., Ramchandani, A., Dong, A., Franco, A., Goyal, A., Saraf, A., Chowdhury, A., Gabriel, A., Bharambe, A., Eisenman, A., Yazdan, A., James, B., Maurer, B., Leonhardi, B., Huang, B., Loyd, B., Paola, B. D., Paranjape, B., Liu, B., Wu, B., Ni, B., Hancock, B., Wasti, B., Spence, B., Stojkovic, B., Gamido, B., Montalvo, B., Parker, C., Burton, C., Mejia, C., Liu, C., Wang, C., Kim, C., Zhou, C., Hu, C., Chu, C.-H., Cai, C., Tindal, C., Feichtenhofer, C., Gao, C., Civin, D., Beaty, D., Kreymer, D., Li, D., Adkins, D., Xu, D., Testuggine, D., David, D., Parikh, D., Liskovich, D., Foss, D., Wang, D., Le, D., Holland, D., Dowling, E., Jamil, E., Montgomery, E., Presani, E., Hahn, E., Wood, E., Le, E.-T., Brinkman, E., Arcaute, E., Dunbar, E., Smothers, E., Sun, F., Kreuk, F., Tian, F., Kokkinos, F., Ozgenel, F., Caggioni, F., Kanayet, F., Seide, F., Florez, G. M., Schwarz, G., Badeer, G., Swee, G., Halpern, G., Herman, G., Sizov, G., Zhang, G., Lakshminarayanan, G., Inan, H., Shojanazeri, H., Zou, H., Wang, H., Zha, H., Habeeb, H., Rudolph, H., Suk, H., Aspegren, H., Goldman, H., Zhan, H., Damlaj, I., Molybog, I., Tufanov, I., Leontiadis, I., Veliche, I.-E., Gat, I., Weissman, J., Geboski, J., Kohli, J., Lam, J., Asher, J., Gaya, J.-B., Marcus, J., Tang, J., Chan, J., Zhen, J., Reizenstein, J., Teboul, J., Zhong, J., Jin, J., Yang, J., Cummings, J., Carvill, J., Shepard, J., McPhie, J., Torres, J., Ginsburg, J., Wang, J., Wu, K., U, K. H., Saxena, K., Khandelwal, K., Zand, K., Matosich, K., Veeraraghavan, K., Michelena, K., Li, K., Jagadeesh, K., Huang, K., Chawla, K., Huang, K., Chen, L., Garg, L., A, L., Silva, L., Bell, L., Zhang, L., Guo, L., Yu, L., Moshkovich, L., Wehrstedt, L., Khabsa, M., Avalani, M., Bhatt, M., Mankus, M., Hasson, M., Lennie, M., Reso, M., Groshev, M., Naumov, M., Lathi, M., Keneally, M., Liu, M., Seltzer, M. L., Valko, M., Restrepo, M., Patel, M., Vyatskov, M., Samvelyan, M., Clark, M., Macey, M., Wang, M., Hermoso, M. J., Metanat, M., Rastegari, M., Bansal, M., Santhanam, N., Parks, N., White, N., Bawa, N., Singhal, N., Egebo, N., Usunier, N., Mehta, N., Laptev, N. P., Dong, N., Cheng, N., Chernoguz, O., Hart, O., Salpekar, O., Kalinli, O., Kent, P., Parekh, P., Saab, P., Balaji, P., Rittner, P., Bontrager, P., Roux, P., Dollar, P., Zvyagina, P., Ratanchandani, P., Yuvraj, P., Liang, Q., Alao, R., Rodriguez, R., Ayub, R., Murthy, R., Nayani, R., Mitra, R., Parthasarathy, R., Li, R., Hogan, R., Battey, R., Wang, R., Howes, R., Rinott, R., Mehta, S., Siby, S., Bondu, S. J., Datta, S., Chugh, S., Hunt, S., Dhillon, S., Sidorov, S., Pan, S., Mahajan, S., Verma, S., Yamamoto, S., Ramaswamy, S., Lindsay, S., Lindsay, S., Feng, S., Lin, S., Zha, S. C., Patil, S., Shankar, S., Zhang, S., Zhang, S., Wang, S., Agarwal, S., Sajuyigbe, S., Chintala, S., Max, S., Chen, S., Kehoe, S., Satterfield, S., Govindaprasad, S., Gupta, S., Deng, S., Cho, S., Virk, S., Subramanian, S., Choudhury, S., Goldman, S., Remez, T., Glaser, T., Best, T., Koehler, T., Robinson, T., Li, T., Zhang, T., Matthews, T., Chou, T., Shaked, T., Vontimitta, V., Ajayi, V., Montanez, V., Mohan, V., Kumar, V. S., Mangla, V., Ionescu, V., Poenaru, V., Mihailescu, V. T., Ivanov, V., Li, W., Wang, W., Jiang, W., Bouaziz, W., Constable, W., Tang, X., Wu, X., Wang, X., Wu, X., Gao, X., Kleinman, Y., Chen, Y., Hu, Y., Jia, Y., Qi, Y., Li, Y., Zhang, Y., Zhang, Y., Adi, Y., Nam, Y., Wang, Y., Zhao, Y., Hao, Y., Qian, Y., Li, Y., He, Y., Rait, Z., DeVito, Z., Rosnbrick, Z., Wen, Z., Yang, Z., Zhao, Z., and Ma, Z. The Llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

Inan, H., Upasani, K., Chi, J., Rungta, R., Iyer, K., Mao, Y., Tontchev, M., Hu, Q., Fuller, B., Testuggine, D., and Khabsa, M. Llama Guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*, 2023.

Juravsky, J., Brown, B., Ehrlich, R., Fu, D. Y., Ré, C., and Mirhoseini, A. Hydragen: High-throughput LLM inference with shared prefixes. *arXiv preprint arXiv:2402.05099*, 2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286. PMLR, 2023.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

Li, J., Wang, M., Zheng, Z., and Zhang, M. LooGLE: Can long-context language models understand long contexts? In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16304–16333, 2024.

Lin, J., Diesendruck, M., Du, L., and Abraham, R. BatchPrompt: Accomplish more with less. In *International Conference on Learning Representations*, pp. 21590–21612, 2024.

Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., and Zhu, C. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 2511–2522, 2023.

Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Zhang, Z., Wong, R. Y. Y., Zhu, A., Yang, L., Shi, X., et al. SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pp. 932–949, 2024.

Pan, Z., Ding, Y., Guan, Y., Wang, Z., Yu, Z., Tang, X., Wang, Y., and Ding, Y. FastTree: Optimizing attention kernel and runtime for tree-structured LLM inference. In *Proceedings of Machine Learning and Systems*, 2025.

Pang, R. Y., Parrish, A., Joshi, N., Nangia, N., Phang, J., Chen, A., Padmakumar, V., Ma, J., Thompson, J., He, H., and Bowman, S. R. QuALITY: Question answering with long input texts, yes! In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 5336–5358, 2022.

Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Rajpurkar, P., Zhang, J., Lopyrev, K., and Liang, P. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 2383–2392, 2016.

Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Ferrer, C. C., Grattafiori, A., Xiong, W., Défossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., and Synnaeve, G. Code Llama: Open foundation models for code, 2024. URL https://arxiv.org/abs/2308.12950.

Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Chen, B., Liang, P., Ré, C., Stoica, I., and Zhang, C. FlexGen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning*, pp. 31094–31116. PMLR, 2023.

Sun, H., Chen, Z., Yang, X., Tian, Y., and Chen, B. TriForce: Lossless acceleration of long sequence generation with hierarchical speculative decoding. In *Conference on Language Modeling*, 2024.

Tillet, P., Kung, H.-T., and Cox, D. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019.

Tolstoy, L. *War and Peace*. 1869.

Wang, S., Liu, Y., Xu, Y., Zhu, C., and Zeng, M. Want to reduce labeling cost? GPT-3 can help. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pp. 4195–4205, 2021.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, pp. 38–45, 2020.

Wu, S., Irsoy, O., Lu, S., Dabravolski, V., Dredze, M., Gehrmann, S., Kambadur, P., Rosenberg, D., and Mann, G. BloombergGPT: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023.

Xia, M., Gao, T., Zeng, Z., and Chen, D. Sheared LLaMA: Accelerating language model pre-training via structured pruning. In *International Conference on Learning Representations*, 2024.

Yao, J., Chen, K., Zhang, K., You, J., Yuan, B., Wang, Z., and Lin, T. DeFT: Decoding with flash tree-attention for efficient tree-structured LLM inference. In *International Conference on Learning Representations*, 2025.

Ye, L., Tao, Z., Huang, Y., and Li, Y. ChunkAttention: Efficient self-attention with prefix-aware KV cache and two-phase partition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 11608–11620, 2024.

Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., and Ceze, L. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of Machine Learning and Systems*, 2025.

Yu, Y., Wang, W., Chen, R., and Pei, J. Accelerate parallelizable reasoning via parallel decoding within one sequence. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 9018–9025, 2025.

Zhao, S., Israel, D. M., Van den Broeck, G., and Grover, A. Prepacking: A simple method for fast prefilling and increased throughput in large language models. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics*, volume 258 of *PMLR*, pp. 4393–4401, 2025.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36, pp. 46595–46623, 2023.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, pp. 62557–62583, 2024.

Zheng, Z., Ji, X., Fang, T., Zhou, F., Liu, C., and Peng, G. BatchLLM: Optimizing large batched LLM inference with global prefix sharing and throughput-oriented token batching. In *Proceedings of Machine Learning and Systems*, volume 8, pp. 1742–1756, 2026.

Zhu, L., Wang, X., Zhang, W., and Lau, R. W. H. RelayAttention for efficient large language model serving with long system prompts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 4945–4957, 2024.

Table 7: SQuAD v1.1 dev (10,570 questions, 48 articles split into 71 chunks of at most 8k tokens) on SGLang v0.5.22 with CodeLlama-7B-Instruct on an RTX A6000. Answer time excludes document prefill for all configurations. ΔF1 is PP minus stock, with a paired bootstrap 95% CI over questions.

Config	EM	F1	ΔF1 [95% CI]	Time (s)
PP (P=32)	21.51	45.14	–	237.6
FlashInfer	21.50	45.25	−0.11 [−.25, .03]	406.5
Triton	21.55	45.18	−0.04 [−.20, .12]	409.2

A Additional SGLang Results

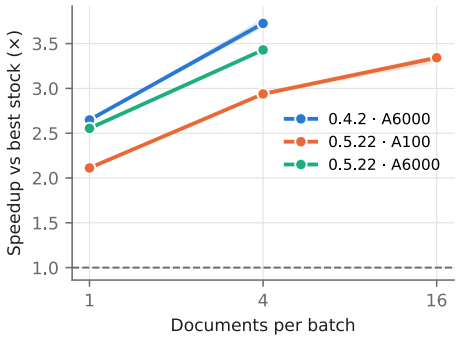


Figure 6: Speedup of PP over the faster stock configuration as more documents share a batch (4k-token documents, 64 questions each, 128 output tokens). Sixteen documents do not fit in the A6000's KV pool.

Figure 6 plots the multi-document results of subsection 5.3.

B Answer Quality

With exact arithmetic, Equation 2 and per-stream positions give each stream exactly the attention inputs of a single-question request. In bf16, however, the different packing changes the order of floating-point reductions.

To measure task quality, we run SQuAD v1.1 (Rajpurkar et al., 2016) through the SGLang implementation. Each Wikipedia article is the shared document, split into chunks of at most 8k tokens (5.5k on average), and its questions become the streams: 149 per chunk on average and up to 766. All configurations use the same instruction prompt, greedy decoding and at most 32 answer tokens. Table 7 shows that PP's EM and F1 match both stock backends: every paired confidence interval on the difference contains

zero. On this realistic workload, with documents shorter than in subsection 5.2, PP generates the answers 1.71× faster. Absolute EM is low for every configuration because the model answers with a short sentence that contains the span rather than the bare span.

On our HuggingFace prototype with Llama 3–8B, PP matches standard prompting's F1 on SQuAD (Rajpurkar et al., 2016), QuAC (Choi et al., 2018) and DROP (Dua et al., 2019), while batch prompting loses accuracy (Table 9 in Appendix D); on QuAC, the prototype scores only the 81% of questions whose dialogue history fits in memory.

C Theoretical Analysis

This appendix analyzes the efficiency gains of Parallel Prompting in LLM inference. We first recall Amdahl's Law for parallel algorithms and then derive the speedup and throughput improvements of our approach.

Amdahl's Law (Amdahl, 1967) gives the speedup of a task when a portion of it is parallelized:

$$S(n) = \frac{1}{(1-p) + \frac{p}{n}} \tag{4}$$

where $S(n)$ is the speedup with n processors, p is the fraction of the task that can be parallelized, $1-p$ is the fraction that remains serial. The overall speedup is therefore limited by the serial portion of the task. As n increases, the speedup approaches $\frac{1}{1-p}$, so returns diminish unless p is close to 1.

In LLM inference, traditional methods process each query sequentially, which is inefficient because decoding processing is serial.Parallel Prompting decodes multiple queries in one sequence, so attention over the shared prefix becomes a matrix-matrix product. We measure throughput as the number of queries (prompts) processed, each to a full output completion, per unit time.

Proposition 1. *Consider inference on N independent queries using (a) standard batch processing and (b) parallel prompting (packing all queries as independent subsequences in a single sequence with attention masking).*

Let $T_{batch} = T_{setup} + N \cdot T_{\mathrm{MV}}$ be the wall-time for a batch (with matrix-vector attention), and $T_{parallel} = T_{setup} + T_{\mathrm{MM}}$ for parallel prompting (with matrix-matrix attention). Then, the respective throughput values are:

$$\mathit{Throughput}_{batch} = \frac{N}{T_{batch}}, \tag{5}$$

$$\mathit{Throughput}_{parallel} = \frac{N}{T_{parallel}} \tag{6}$$

and

$$\frac{\mathit{Throughput}_{parallel}}{\mathit{Throughput}_{batch}} = \frac{T_{batch}}{T_{parallel}} = \frac{T_{setup} + NT_{\mathrm{MV}}}{T_{setup} + T_{\mathrm{MM}}} \tag{7}$$

where T_{MV} is per-query wall-time for the matrix-vector attentions, and T_{MM} is wall-time for the matrix-matrix product in the attention.

Since $T_{\mathrm{MM}} \geq T_{\mathrm{MV}}$, the ratio is at most N, and it approaches this upper bound only if T_{setup} is negligible and T_{MM} stays close to T_{MV}.

Assumptions.

- The model and hardware support this masking and packing; T_{MV} and T_{MM} are measured compatibly.
- Setup time is equal for standard batch processing and parallel prompting.
- N is small enough to avoid exceeding hardware or memory limits for both methods.

In practice T_{MM} grows with N: each stream also scans its sibling streams' masked keys (subsection 5.3). The measured gains, at most 8.5× (section 5), are therefore far below N, and the bound is not attained.

C.1 Throughput Maximization by Balancing Attention Parallelism and Batch Size

Batching is an important technique for raising throughput in LLM inference. In batched decoding, each forward pass processes the latest token of many sequences instead of one. This raises the arithmetic intensity of transformer components such as the multilayer perceptron (MLP) blocks and lets them use hardware-friendly matrix multiplications.

The arithmetic intensity of attention does not benefit from batching, because each sequence has its own key and value matrices. While the other components use tensor cores during batched decoding, attention must still be computed as many independent matrix-vector products. Parallel Prompting aims to raise the arithmetic intensity of attention.

Proposition 2 (Throughput Maximization)**.** *Let P be the number of streams per request (number of independent queries packed into a sequence for matrix-matrix attention), B the batch size (number of such sequences processed in parallel), and $P \cdot B \leq S^*$ a hardware resource constraint (e.g., total token capacity).*

Let $T_{\mathrm{attn}}(P)$ denote the attention computation cost (function of P), and $T_{\mathrm{mlp}}(PB)$ denote the MLP/other cost (a function of the PB tokens per step).

Then, the throughput (queries per unit time) satisfies:

$$\mathrm{Throughput}(P, B) = \frac{P \cdot B}{T_{\mathrm{attn}}(P) + T_{\mathrm{mlp}}(PB)} \tag{8}$$

and maximal throughput is achieved at

$$(P^*, B^*) = \arg\max_{P \cdot B \le S^*} \frac{P \cdot B}{T_{\text{attn}}(P) + T_{\text{mlp}}(PB)} \quad (9)$$

where the per-stream attention cost $T_{\text{attn}}(P)/P$ generally falls with P up to a hardware limit (then rises), and $T_{\text{mlp}}(PB)$ grows sublinearly in PB up to a limit.

Assumptions.

- P queries are packed per prompt and B prompts per batch, with $PB \le S^*$ (a resource or hardware constraint).
- $T_{\text{attn}}(P)$ and $T_{\text{mlp}}(PB)$ are the attention and MLP wall times per step.
- The per-token costs $T_{\text{attn}}(P)/P$ and $T_{\text{mlp}}(PB)/(PB)$ are nonincreasing up to hardware limits, and nonmonotone beyond them.

The maximizing pair (P^*, B^*) is found by balancing optimal matrix-matrix utilization for attention and optimal batch size for MLP efficiency. We expect the throughput function to be quasi-concave in (P, B) under natural hardware scaling assumptions for transformer kernels; we do not prove this, but the U-shaped wall-time curves in Figure 5 are consistent with it. The theoretical maximum could exist at an interior point determined by hardware and model specifics, and is not achieved by maximizing either P or B alone.

D Further Results of the HuggingFace Prototype

This appendix gives further results from our first implementation of Parallel Prompting, built on HuggingFace Transformers (Wolf et al., 2020) (subsection 5.7).

We evaluate the prototype in two sets of experiments: (1) controlled scaling studies on small and medium-sized models using synthetic data, and (2) a downstream task evaluation on reading comprehension datasets using Llama 3–8B. All experiments run on a single NVIDIA A100-80GB GPU with PyTorch implementations built on HuggingFace Transformers (Wolf et al., 2020).

D.1 Scaling Experiments

Setup. Following Juravsky et al. (2024), we construct synthetic datasets with varying document lengths, numbers of unique documents, and numbers of queries. Document content is drawn from a subset of *War and Peace* (Tolstoy, 1869). We perform all scaling studies on CodeLlama-7B-Instruct (Rozière et al., 2024) and Sheared-LLaMA-1.3B (Xia et al., 2024) to enable controlled analysis under constrained compute.

Table 8: Synthetic throughput (tokens/s) of the prototype and Hydragen, 8 documents, CodeLlama-7B-Instruct, A100-80GB.

Questions	Method	512	1024	2048
8×128	Hydragen	2479	2161	1743
8×128	Ours	2528	1995	1355

Memory Constraints and Throughput Under Increasing Context Length. We first examine memory usage and throughput as the number of queries and the shared-context length increase. Table 8 summarizes the results. Throughput falls with longer documents for both methods. The prototype matches Hydragen at 512 tokens, but Hydragen's dedicated prefix kernel is faster at 1024 and 2048 tokens, which motivates moving Parallel Prompting into an engine with fused attention kernels (section 4).

D.2 Case Study: Question Answering Performance

We evaluate our method on downstream reading comprehension tasks to measure its end-to-end effect on answer quality and generation speed. We use Llama 3–8B (Grattafiori et al., 2024) and measure F1 scores (standard for QA) on SQuAD (Rajpurkar et al., 2016), QuAC (Choi et al., 2018), and DROP (Dua et al., 2019).

As shown in Table 9, Parallel Prompting keeps the F1 of standard prompting and is 3.5×, 8.1× and 5.5× faster on SQuAD, QuAC and DROP, while sequential batch prompting loses F1. On QuAC, the Parallel time covers only the 5,951 of 7,354 questions whose dialogue history fits in memory, so the QuAC ratio overstates the like-for-like speedup.

E Additional Prototype Results

We use smaller models and synthetic data for the analytical and ablation studies than for the downstream evaluations. Large models such as Llama 3-8B are computationally expensive, which makes extensive ablation and scaling experiments over many parameters hard to run. Smaller models and synthetic datasets let us vary batch size, prefix length and number of queries systematically and isolate the effect of our method under controlled conditions. The synthetic studies show how throughput and memory change with batch size, prefix length and number of queries, and the downstream benchmarks measure end-to-end quality and speed.

Memory Usage Table 10 shows that the Parallel method uses less GPU memory than standard prompting on all three datasets. Its memory use depends on the maximum number of questions per batch, which we set as large as memory allows. Reducing it on QuAC lowers memory further while

Table 9: Comparison of generation time and performance for downstream tasks with different methods with Llama 3 8B model on A100-80G. Standard, SeqBatch and Parallel are rerun with the original scripts; Hydragen and vLLM[0.6.4] are from the original runs. subsection 5.2 compares against current SGLang. Std is the across-run standard deviation of the time where available. *QuAC Parallel and SeqBatch score only the questions that fit in memory (5,951 and 2,899 of 7,354). F1 is computed as the harmonic mean of precision and recall in extractive QA.

Method	SQuAD			QuAC			DROP		
	Times(s)	Std	F1(%)	Time(s)	Std	F1(%)	Time(s)	Std	F1(%)
Standard	597	–	87.2	1937	–	34.0	647	–	58.1
SeqBatch	365	–	84.3	346*	–	32.4*	843	–	42.6
Hydragen	1651	20.9	87.1	1230	6.74	34.0	471	3.85	58.2
vLLM	373	0.46	87.4	897	0.57	32.8	417	0.44	58.5
Parallel	169	–	87.2	238*	–	34.2*	117	–	58.2

keeping a large speedup over the baseline. Table 6 lists memory, time and F1 for several values.

Effect of the Number of Questions. We sweep over the number of queries for fixed document and query lengths. Table 11 shows that throughput improves as the number of parallel queries increases, more for the 1.3B model than for the 7B model. At small batch sizes, non-attention operations dominate; at large query counts we expect attention over long prefixes to become the bottleneck, and parallel decoding to give larger gains there.

Sequence Length vs. Computation Gains Trade-off. Our measurements show that throughput increases with batch size or streams per request up to a point, after which the overhead of the longer packed input sequences outweighs the advantage of matrix-matrix attention, as 2 predicts. In our SGLang measurements, $P = 16$–32 is optimal, and $P = 64$ in one setting (subsection 5.4).

Compatibility with Speculative Decoding. Parallel Prompting decodes multiple suffixes in lock-step to generate answers to several queries at once, while speculative decoding verifies tokens of a single sequence. The two are distinct but may be complementary: speculative decoding could run within each Parallel Prompting stream, or be adapted to verify multiple shared-prefix continuations in parallel.

Developer Overhead and Practical Adoption. In many production stacks, the shared-prefix boundary is already explicit: for example, retrieval-augmented generation (RAG) pipelines concatenate retrieved context (prefix) with a question (suffix), and batched APIs group queries under a common header or instruction. In these settings, Parallel Prompting needs only two inputs: (1) the token span (or delimiter) of the shared prefix and (2) a list of per-query suffixes. Our current engine supports only offline workloads (section 7); online serving needs scheduler support.

Effect of Model Size We also vary document length for CodeLlama-7b-Inst (Rozière et al., 2024) and Sheared-LLaMA-1.3B (Xia et al., 2024) (Table 12). We therefore also run several configurations with CodeLlama-7b-Inst (Rozière et al., 2024) and Sheared-LLaMA-1.3B (Xia et al., 2024). See Table 12 for results.

Table 10: Comparison of memory usage with different methods with Llama 3 8B model on A100-80G. These runs are a separate run set from Table 9, so their times differ. Memory is in GiB; the SQuAD values were converted from 55,700 and 48,600 MiB.

Dataset	Method	Time(s)	Memory(GiB)
SQuAD	Standard	590	54.4
	Parallel	168	47.5
QuAC	Standard	1799	53.6
	Parallel	352	33.1
DROP	Standard	654	54.3
	Parallel	111	36.1

Table 11: Throughput (tokens/sec) under different batch sizes for parallel generation with Sheared-LLaMA-1.3B and CodeLlama-7B when $doc_len = 512$, $q_len = 12$, and $ans_len = 5$.

Num_Questions	Batch Size	Throughput-1.3B	Throughput-7B
128	1	4283	1931
	2	4625	1843
	4	3654	1468
	8	2850	1018
256	1	5911	2115
	2	6384	2250
	4	5748	2071
	8	4959	1615
512	1	5419	1850
	2	6845	2214
	4	7725	2382
	8	7181	2146

Table 12: Comparing the throughput using parallel Batching with CodeLlama-7B and Sheared-LLaMA-1.3B with different lengths of doc length when $q_len = 12 \| q_num = 128 \| ans_len = 5$ and the number of unique doc content equals 8. Throughput falls by about 30% for both models as the document length grows from 256 to 1024 tokens.

doc_len	$Throughput(1.3B)(tokens/second)$	$Throughput(7B)(tokens/second)$
256	9514	2751
512	8199	2430
1024	6592	1924